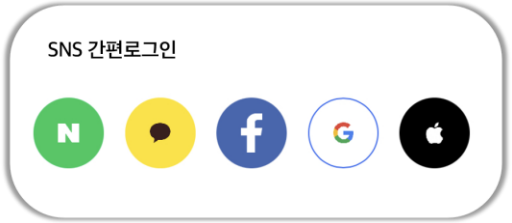
웹 개발자 부트캠프 과정

SeSAC x CODINGOn

OAuth

OAuth란?

OAuth(Open Authorization)



- 인터넷 사용자들이 비밀번호를 직접 제공하지 않고 다른 웹사이트 상의 자신들의 정보에 대해 웹사이트나 어플리케이션의 접근 권한을 부여할 수 있는 공통적인 수단으로서 사용되는, 접근 위임을 위한 개방형 표준이다(위키백과)

- 즉, 서드파티 어플리케이션이 사용자의 계정에 접근할 수 있는 권한을 부여하기 위한 프로토콜

OAuth

OAuth 1.0

- 서드파티 어플리케이션이 사용자의 데이터에 접근할 권한을 얻기 위해 **서명**된 요청을 사용
- 요청의 **서명** 부분은 어플리케이션의 비밀 키와 사용자의 토큰을 조합하여 생성되며, 이를 통해 **보안 유지**

OAuth 2.0

- OAuth 2.0은 보다 간편하고 확장성이 있는 버전으로, 기본적으로 **인증과 권한을 분리**하여 다루는 것을 특징
- **Access Token**을 통해 권한을 부여하고, 사용자의 실제 비밀 정보를 공유하지 않습니다.

OAuth 2.0

JWT Bearer Token 인증

- OAuth 2.0에서는 Bearer Token 인증 방식을 지원
- JWT를 Bearer Token으로 사용할 수 있음
- 클라이언트가 발급받은 JWT를 "Authorization" 헤더에 담아 요청을 보내면, 서버는 해당 JWT의 유효성을 검사하여 권한을 부여

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjE
```

* Bearer? 웹 서비스에서 사용자 인증에 사용되는 인증 방식 중 하나

JWT

JWT란?

JWT(JSON Web Token)

- 웹 애플리케이션에서 정보를 안전하게 전송하기 위한 간단한 방법
- 인증된 사용자를 식별하거나 데이터에 서명을 하여 변조를 방지하는 데 사용

JWT 구성

- **Header**: 토큰의 타입과 해시 알고리즘 정보가 포함
- **Payload**: 실제 정보 데이터가 포함되며, 클레임(claim)이라 불리는 키-값 쌍으로 이루어져 있음
- **Signature**: 토큰의 무결성을 검증하기 위한 서명 부분으로, Header와 Payload의 조합에 비밀 키를 사용해 생성

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "Damon",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkRhbW9uIiwiaWF0IjoxNTE2MjM5MDIyfQ.yMVWujYwc1Kpi-A2BWH7pQNgKP2QMPaPqH6W93Azwio

JWT 구성

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "Damon",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

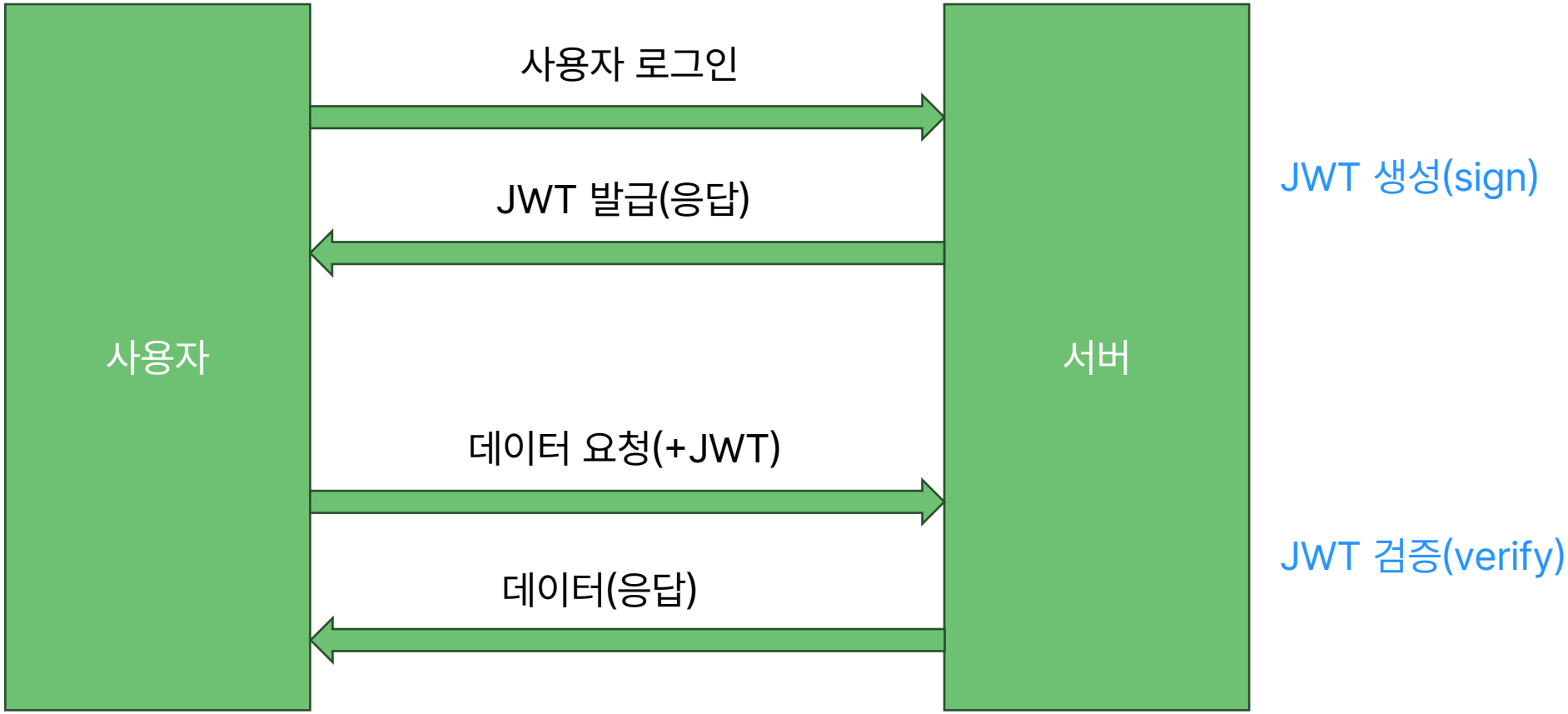
```
"alg" : "HS256" // JWT 서명 알고리즘, HMAC SHA-256을 사용하여 서명 생성.
"typ" : "JWT"   // 토큰 유형이 JWT 임을 명시.

"sub" : "~~"    // "sub"(subject) 클레임으로, JWT의 주체를 나타냄. (사용자 ID)
"name" : "~~"   // "name" 클레임으로, 사용자의 이름을 나타냄.
"iat" : ~~      // "iat"(Issued At) 클레임으로, JWT가 발급된 시간을 타임스탬프 형식으로 나타냄.

HMACSHA256(...): // HMAC SHA-256 알고리즘을 사용하여 서명 생성 과정
base64UrlEncode(header) ~ "." ~(payload) : // 헤더와 페이로드를 Base64 URl 인코딩 한 후 . 결합

your-256-bit-secret: // 서명 생성 시 사용되는 비밀 키. 서버에서만 알고 있어야 함.
```

JWT 작동방식


사용자
서버
사용자 로그인
JWT 발급(응답)
데이터 요청(+JWT)
데이터(응답)
JWT 생성(sign)
JWT 검증(verify)

Key Point

- JWT는 ***서명(인증)***을 목적으로 사용.
  JWT는 Base64로 **인코딩**되어 있지만, 이는 **암호화**가 아니라 단순히 **인코딩**이기 때문에,
  디버거를 사용하면 페이로드의 내용을 쉽게 **복호화**할 수 있음.
  따라서, 페이로드에는 **비밀번호**와 같은 **민감한 정보**를 포함하면 안 됨.

- JWT의 **주요 목적**은 **정보 보호**가 아니라 **위조 방지**이다.
  서명을 통해 데이터의 **무결성**을 보장하며, **비밀키**가 노출되지 않는 한,
  토큰의 내용이 위조되더라도 서명을 통해 **위조된 토큰**을 걸러낼 수 있다.

JWT 주요함수

- sign: 서버측에서 JWT를 생성할 때 사용

```
jwt.sign(payload, secretOrPrivateKey, [options, callback])
```

- verify: 클라이언트나 서버에서 받은 JWT의 유효성을 검증할 때 사용

```
jwt.verify(token, secretOrPublicKey, [options, callback])
```

JWT 사용하기

```
npm install jsonwebtoken
```

```
const jwt = require('jsonwebtoken');
```